A Guide to the Stellarator Design Workflow

stellarator_workflow repository draft

March 11, 2026

Abstract

This document explains how a modern stellarator workflow turns a prescribed plasma boundary and profile guesses into equilibrium solutions, transport predictions, fusion-power estimates, and optimization metrics. The focus is the practical code chain built around VMEC or DESC for equilibrium, BOOZ_XFORM for Boozer coordinates, NEO / SFINCS / MONKES for neoclassical and orbit-space metrics, GENE or GX for turbulence, and Trinity3D or NEOPAX for profile evolution. The main objective is to make the data hand-offs between these codes explicit, so that the physics workflow can also be understood as a software workflow suitable for reproducible, containerized execution.

Contents

1 Fusion energy, stellarators, and optimization **2**

2 Inputs, state variables, and final performance metrics **2**

3 The workflow as a data pipeline **3**

4 Code-by-code details **5**

4.1 VMEC and vmec_jax: ideal-MHD equilibrium 5

4.2 DESC: differentiable pseudo-spectral equilibrium 6

4.3 BOOZ_XFORM and booz_xform_jax: VMEC to Boozer coordinates 7

4.4 NEO and NEO_JAX: effective ripple in the $1/\nu$ regime 8

4.5 SFINCS and sfincs_jax: full neoclassical drift-kinetic transport 8

4.6 MONKES: monoenergetic neoclassical coefficients 9

4.7 GENE, GENE-3D, GX, and SPECTRAX-GK: turbulent transport 10

4.8 Trinity3D: whole-device profile evolution 10

4.9 NEOPAX: a reduced JAX neoclassical transport loop 11

5 Which quantities are optimization targets and which are transport inputs? **12**

6 Why the final temperature and density profiles differ from the initial guesses 12

7 Examples from the literature 13

8 What a future containerized black-box workflow should standardize 13

1 Fusion energy, stellarators, and optimization

Fusion energy aims to extract useful power from reactions between light nuclei, most commonly deuterium and tritium. In a magnetic-confinement concept, the plasma must be kept hot, dense, and well confined long enough for fusion heating to offset the losses. Stellarators are one route to that goal: they use fully three-dimensional magnetic fields generated mainly by external coils, which reduces reliance on a large toroidal plasma current and makes steady-state operation especially attractive [7, 14].

That same three-dimensional freedom is also what makes stellarator design a high-dimensional optimization problem. The boundary shape that creates nested flux surfaces also changes trapped-particle orbits, neoclassical transport, turbulence, bootstrap current, and MHD stability. In other words, a stellarator is not judged only by whether an equilibrium exists; it is judged by how that equilibrium performs after the whole transport and power-balance loop is considered [1, 18, 19].

Designing a stellarator is therefore not a single computation. It is an iterative chain:

1. choose a plasma boundary and rough pressure / current profile,
2. solve the three-dimensional magnetohydrodynamic equilibrium,
3. transform the equilibrium into coordinates suitable for transport and orbit calculations,
4. compute neoclassical and turbulent transport,
5. evolve the density and temperature profiles,
6. feed the updated pressure and current back into the equilibrium calculation,
7. evaluate whether the resulting device has acceptable confinement, transport, and fusion power.

The most important conceptual point is that the profile used to *start* the equilibrium solve is usually not the profile used to quote the device's final performance. The final performance comes from the transport-consistent profiles after the loop closes.

2 Inputs, state variables, and final performance metrics

The workflow begins with geometry and profile inputs, but it ends with a much broader set of performance metrics. The role of the intermediate codes is to propagate information from one stage of physics to the next while preserving enough detail to support optimization.

We assume stellarator symmetry, so the boundary can be written using only cosine R_{mn} and sine Z_{mn} terms:

$$R(\theta, \phi) = \sum_{m,n} R_{mn} \cos(m\theta - nN_{\text{fp}}\phi)\,, \tag{1}$$

$$Z(\theta, \phi) = \sum_{m,n} Z_{mn} \sin(m\theta - nN_{\text{fp}}\phi)\,. \tag{2}$$

In VMEC input files these are usually represented as `RBC(m,n)` and `ZBS(m,n)`. In DESC the same physical surface is stored in its own spectral basis.

The remaining inputs are profile guesses:

- a pressure profile p,
- either rotational transform ι or a current profile,
- a total toroidal flux / magnetic-field scale.

The converged output of the full workflow is not just "an equilibrium". It is the combination of:

- geometry metrics: major radius R, minor radius a, aspect ratio R/a, volume, β, ι,
- Boozer-space metrics: symmetry-breaking harmonics, $I(\psi)$, $G(\psi)$, trapped-particle proxies,
- neoclassical metrics: ϵ_{eff}, bootstrap current, ambipolar E_r, neoclassical particle and heat fluxes,
- turbulence metrics: linear growth rates, nonlinear heat fluxes, stiffness,
- whole-device metrics: transport-consistent $n(r)$ and $T(r)$, fusion power, auxiliary power, and $Q = P_{\text{fus}}/P_{\text{aux}}$.

When the final species profiles are available, the fusion power is computed from the transport-consistent pressure and composition:

$$P_{\text{fus}} = \int dV\; n_D n_T \langle \sigma v \rangle_{DT} E_{DT}, \tag{3}$$

with $E_{DT} \approx 17.6$ MeV and the thermal reactivity $\langle \sigma v \rangle_{DT}$ typically evaluated using the Bosch-Hale parametrization [3].

3 The workflow as a data pipeline

Table 1 summarizes the software contracts in the order they usually appear in a design loop.

Table 1: Core data contracts in a stellarator design workflow.

Stage	Codes	Typical inputs	Typical outputs	Downstream role / direct objective
Equilibrium	VMEC, vmec_jax, DESC	boundary Fourier coefficients, pressure coefficients, iota or current coefficients, toroidal flux	*wout* or HDF5 equilibrium, R, a, aspect ratio, volume, β, ι, field harmonics	feeds everything downstream; geometry metrics are also direct objectives
Boozer transform	BOOZ_XFORM, booz_xform_jax, DESC Boozer utilities	VMEC- or DESC-like equilibrium	`boozmn`, Boozer B_{mn}, Boozer geometry, $I(\psi)$, $G(\psi)$, angle shifts	feeds NEO, SFINCS, MONKES, some gyrokinetic geometry modules; symmetry-breaking spectrum is a direct objective
Orbit / ripple diagnostics	NEO, NEO_JAX	Boozer Fourier data, surface list, control file	$\epsilon_{\text{eff}}^{3/2}$, trapped-particle class contributions, reference radii	mostly a screening metric or optimization objective; usually not directly advanced by a transport solver
Neoclassical transport	SFINCS, sfincs_jax	Boozer geometry, species gradients, E_r guess, collision model	particle flux, heat flux, bootstrap current, E_r, Φ_1, transport matrices	can feed a transport solver directly; bootstrap current and pressure updates can be sent back to equilibrium
Reduced neoclassical stage	MONKES, NEOPAX	Boozer or DESC geometry, Maxwellians, E_r, monoenergetic scan database	$D_{ij}(r, \nu, E_r)$, thermal L_{ij} matrices, ambipolar E_r, reduced transport evolution	MONKES feeds NEOPAX; both can be used in optimization loops when full SFINCS is too expensive
Turbulence	GENE, GENE-3D, GX, SPECTRAX-GK	geometry, species profiles and gradients, collisions, electromagnetic parameters	growth rates, frequencies, nonlinear heat and particle fluxes, spectra	heat fluxes are both direct objectives and inputs to global transport solvers

Stage	Codes	Typical inputs	Typical outputs	Downstream role / direct objective
Global transport	Trinity3D, NEOPAX	initial profiles, source models, geometry, neoclassical and turbulent fluxes	updated $n(r)$, $T(r)$, $p(r)$, $E_r(r)$, fusion-power metrics, profile histories	closes the loop by producing the new pressure and often new current / bootstrap information

Two practical distinctions are worth making explicitly:

- ϵ_{eff} is extremely useful for optimization, but it is not normally the quantity that Trinity3D advances.
- nonlinear heat flux from GX or GENE is both a design objective and a direct transport input.

4 Code-by-code details

4.1 VMEC and vmec_jax: ideal-MHD equilibrium

What these codes do. VMEC solves the three-dimensional ideal-MHD equilibrium for a prescribed boundary and prescribed pressure / current data. The JAX implementation `vmec_jax` is designed to preserve the same physics and very similar input / output contracts while enabling differentiation and JIT execution [16, 35].

Governing equations. The equilibrium is defined by

$$\nabla p = \mathbf{J} \times \mathbf{B}, \qquad \nabla \cdot \mathbf{B} = 0, \qquad \mathbf{J} = \frac{1}{\mu_0} \nabla \times \mathbf{B}. \tag{4}$$

VMEC uses straight-field-line coordinates. Introducing

$$u = \theta + \lambda(s, \theta, \zeta), \qquad \frac{du}{d\zeta} = \iota(s), \tag{5}$$

the equilibrium is found as a stationary point of the ideal-MHD energy functional [17]:

$$W = \frac{1}{(2\pi)^2} \int \left(\frac{B^2}{2} + \frac{p}{\gamma - 1} \right) dV. \tag{6}$$

In VMEC's flux-coordinate representation, the contravariant field components used in the implementation are [35]:

$$B^\zeta = \frac{\Phi'(s) + \text{lamscale}\, \partial_\theta \lambda}{\text{signgs}\, \sqrt{g}\, 2\pi}, \tag{7}$$

$$B^\theta = \frac{\chi'(s) - \text{lamscale}\, \partial_\zeta \lambda}{\text{signgs}\, \sqrt{g}\, 2\pi}. \tag{8}$$

Inputs. The legacy VMEC input contract is typically:

- boundary coefficients `RBC(m,n)` and `ZBS(m,n)`,
- pressure coefficients `AM`,
- either iota coefficients `AI` or current coefficients `AC`,
- a flux scale such as `PHIEDGE`,
- grid / resolution / convergence controls.

Outputs. The standard product is `wout_*.nc`, which contains the geometry and diagnostic scalars needed by downstream codes: aspect ratio, major radius, minor radius, volume, total beta, iota, pressure profiles, field harmonics, Jacobian information, and real-space geometry on flux surfaces.

How these outputs are used. The equilibrium is the universal upstream state. BOOZ_XFORM uses it directly, GX geometry utilities derive flux-tube metrics from it, and transport solvers use its flux-surface geometry. Many device objectives, such as target aspect ratio or target volume, are already available at this stage.

4.2 DESC: differentiable pseudo-spectral equilibrium

What DESC does. DESC solves the same physical equilibrium problem, but in an inverse pseudo-spectral formulation designed for optimization and automatic differentiation [4–6, 27, 28].

Governing equations. DESC represents the magnetic field in flux coordinates (ρ, θ, ζ) as

$$\mathbf{B} = \frac{\partial_\rho \psi}{2\pi\sqrt{g}} \left[\left(\iota - \frac{\partial \lambda}{\partial \zeta} \right) \mathbf{e}_\theta + \left(1 + \frac{\partial \lambda}{\partial \theta} \right) \mathbf{e}_\zeta \right]. \tag{9}$$

The current density follows from Ampere's law:

$$J^\rho = \frac{\partial_\theta B_\zeta - \partial_\zeta B_\theta}{\mu_0 \sqrt{g}}, \qquad J^\theta = \frac{\partial_\zeta B_\rho - \partial_\rho B_\zeta}{\mu_0 \sqrt{g}}, \qquad J^\zeta = \frac{\partial_\rho B_\theta - \partial_\theta B_\rho}{\mu_0 \sqrt{g}}. \tag{10}$$

Force balance is

$$\mathbf{F} \equiv \mathbf{J} \times \mathbf{B} - \nabla p = 0. \tag{11}$$

In practice DESC solves a collocated nonlinear system in radial and helical components of the force residual.

Inputs and outputs. DESC accepts its own input format or imported VMEC inputs. It requires the boundary, pressure coefficients, iota or current coefficients, and toroidal flux. It outputs an HDF5 equilibrium together with a wide set of geometry and optimization diagnostics.

How DESC fits in the workflow. DESC can replace VMEC as the equilibrium engine, and it is especially useful when the workflow is part of a differentiable optimization loop. DESC can also compute several geometry objectives directly, reducing the number of external post-processing steps needed during optimization.

4.3 BOOZ_XFORM and booz_xform_jax: VMEC to Boozer coordinates

What these codes do. Boozer coordinates are the standard coordinate system for much of stellarator neoclassical analysis. BOOZ_XFORM converts a VMEC-like equilibrium into the Boozer representation [2, 15, 30].

Governing equations. Boozer coordinates satisfy

$$\mathbf{B} = \nabla\psi \times \nabla\theta_B + \iota(\psi)\nabla\zeta_B \times \nabla\psi = \beta\nabla\psi + I(\psi)\nabla\theta_B + G(\psi)\nabla\zeta_B. \tag{12}$$

Let ζ_0 and θ_0 denote the original angle coordinates and define

$$\zeta_B = \zeta_0 + \nu. \tag{13}$$

Then the Boozer poloidal angle is

$$\theta_B = \theta_0 + \lambda + \iota\nu. \tag{14}$$

The angle-shift field ν is determined by the covariant field components:

$$B_{\theta_0} = \left(1 + \frac{\partial\lambda}{\partial\theta_0}\right) I + (G + \iota I)\frac{\partial\nu}{\partial\theta_0}, \tag{15}$$

$$B_{\zeta_0} = G + I\frac{\partial\lambda}{\partial\zeta_0} + (G + \iota I)\frac{\partial\nu}{\partial\zeta_0}. \tag{16}$$

The implementation summarized in the BOOZ_XFORM theory note expresses this as

$$\nu = \frac{w - I\lambda}{G + \iota I}, \tag{17}$$

where the Fourier coefficients of w are reconstructed from the original covariant field harmonics.

Inputs and outputs. The standard inputs are a VMEC `wout` file and a requested Boozer resolution. The standard output is `boozmn_*.nc`, containing Boozer harmonics such as:

- `bmnc_b`: $|B|$ in Boozer coordinates,
- `rmnc_b`, `zmns_b`: geometry in Boozer coordinates,
- `pmns_b`: toroidal angle shift,
- `gmn_b`: Jacobian harmonics,
- `buco_b`, `bvco_b`: Boozer currents.

How these outputs are used. NEO and SFINCS typically expect Boozer-space data. The Boozer spectrum is also one of the cleanest direct measures of quasi-symmetry.

4.4 NEO and NEO_JAX: effective ripple in the $1/\nu$ regime

What these codes do. NEO evaluates effective helical ripple and related trapped-particle quantities from Boozer-coordinate equilibria [26, 29, 31].

Governing equations. The field-line ODE accumulates the integrals

$$y_2 = \int d\phi\, B^{-2}, \qquad y_3 = \int d\phi\, |\nabla\psi| B^{-2}, \qquad y_4 = \int d\phi\, K_G B^{-3}, \tag{18}$$

along with pitch-angle dependent trapped-particle integrals such as

$$I_f = \int d\phi \sqrt{1 - \frac{B}{B_0\eta}}\, B^{-2}, \tag{19}$$

$$H_f = \int d\phi \sqrt{1 - \frac{B}{B_0\eta}} \left(\frac{4}{B/B_0} - \frac{1}{\eta} \right) \frac{K_G}{\sqrt{\eta}} B^{-2}. \tag{20}$$

For each trapped-particle class,

$$\epsilon_{\text{eff}}^{3/2}(m) = C_\epsilon \frac{y_2}{y_3^2}\, \text{BigInt}(m), \qquad C_\epsilon = \frac{\pi R_0^2 \Delta\eta}{8\sqrt{2}}. \tag{21}$$

The total `epstot` is the sum over classes.

Inputs and outputs. The usual inputs are `boozmn_*.nc` and a `neo_in.*` control file. The output `neo_out.*` contains ϵ_{eff}, surface radii, iota, and optional current-related diagnostics.

How these outputs are used. These quantities are often used as optimization metrics or rapid screening diagnostics. They usually do not directly enter a profile evolution equation in Trinity3D.

4.5 SFINCS and sfincs_jax: full neoclassical drift-kinetic transport

What these codes do. SFINCS solves the stellarator neoclassical drift-kinetic equation with realistic collisions and, optionally, a poloidally varying electrostatic potential [20, 21, 25, 33].

Governing equations. One common form of the first-order kinetic equation solved by SFINCS is

$$\left(v_\parallel \mathbf{b} + \frac{d\Phi_0}{dr} \frac{\mathbf{B} \times \nabla r}{B^2} \right) \cdot \nabla f_{s1} + \left[-\frac{(1-\xi^2)v}{2B} \nabla_\parallel B + \cdots \right] \frac{\partial f_{s1}}{\partial \xi} - (\mathbf{v}_{ms} \cdot \nabla r) \frac{Z_s e}{2T_s x_s} \frac{d\Phi_0}{dr} \frac{\partial f_{s1}}{\partial x_s}$$
$$+ (\mathbf{v}_{ms} \cdot \nabla r) \left[\frac{1}{n_s} \frac{dn_s}{dr} + \frac{Z_s e}{T_s} \frac{d\Phi_0}{dr} + \left(x_s^2 - \frac{3}{2} \right) \frac{1}{T_s} \frac{dT_s}{dr} \right] f_{sM} = C_s[f_{s1}] + S_s. \tag{22}$$

The linearized collision operator is

$$C_s[f_s] = \sum_b C^l_{sb}[f_s, f_b], \qquad C^l_{ab} = C^L_{ab} + C^E_{ab} + C^F_{ab}. \tag{23}$$

When Φ_1 is included, the system is coupled to quasineutrality:

$$\lambda + \sum_s Z_s \int d^3v \, (f_{s0} + f_{s1}) = 0, \tag{24}$$

together with moment and gauge constraints such as $\langle \Phi_1 \rangle = 0$.

Inputs. SFINCS typically needs:

- Boozer geometry (from `boozmn` or `.bc`),
- species masses, charges, densities, temperatures, and radial gradients,
- a radial electric-field guess,
- collision-operator and numerical-resolution choices.

Outputs. The standard output file `sfincsOutput.h5` includes particle flux, heat flux, bootstrap current, flow moments, transport matrices, and optional Φ_1 diagnostics.

How these outputs are used. In a reduced workflow, SFINCS can directly provide the neoclassical fluxes used in a global transport solve. Bootstrap current and ambipolar E_r can also be fed back to the equilibrium stage.

4.6 MONKES: monoenergetic neoclassical coefficients

What MONKES does. MONKES is a fast solver for monoenergetic neoclassical transport coefficients, designed to play a role similar to DKES but with improved modern numerics [8, 9].

Governing equations. In the JAX port used here, the Legendre-expanded pitch-angle operators are

$$L_k(f) = \frac{k}{2k-1}\left[\mathbf{b}\cdot\nabla f + \frac{k-1}{2}\frac{\mathbf{b}\cdot\nabla B}{B} f\right], \tag{25}$$

$$U_k(f) = \frac{k+1}{2k+3}\left[\mathbf{b}\cdot\nabla f - \frac{k+2}{2}\frac{\mathbf{b}\cdot\nabla B}{B} f\right], \tag{26}$$

$$D_k(f) = -\frac{\hat{E}_r}{\psi_r \langle B^2 \rangle}\mathbf{B}\times\nabla\psi\cdot\nabla f + \frac{k(k+1)}{2}\hat{\nu} f. \tag{27}$$

The monoenergetic coefficients are then assembled from flux-surface averages of the solved response:

$$D_{ij} = \begin{pmatrix} D_{11} & D_{12} & D_{13} \\ D_{21} & D_{22} & D_{23} \\ D_{31} & D_{32} & D_{33} \end{pmatrix}. \tag{28}$$

Inputs and outputs. The code takes a single-surface field, species Maxwellians, a radial electric field, a speed, and numerical resolution. The main output is D_{ij}, together with the perturbed distribution and source vectors.

How MONKES fits in the workflow. Its main downstream role is the creation of the database $D_{ij}(r, \nu, E_r)$ for NEOPAX.

4.7 GENE, GENE-3D, GX, and SPECTRAX-GK: turbulent transport

What these codes do. These codes solve the gyrokinetic equations, which are the standard reduced kinetic model for low-frequency plasma turbulence. GENE and GENE-3D are high-fidelity grid-based Eulerian tools; GX uses a GPU-native Fourier–Hermite–Laguerre representation [10–12, 22–24, 34, 36].

Governing equations. A generic δf gyrokinetic equation can be written

$$\frac{\partial h_s}{\partial t} + v_\parallel \mathbf{b} \cdot \nabla h_s + \mathbf{v}_{Ds} \cdot \nabla h_s + \mathbf{v}_E \cdot \nabla h_s - C[h_s] = -\frac{Z_s e F_{Ms}}{T_s} \frac{\partial \langle \chi \rangle}{\partial t} - \mathbf{v}_\chi \cdot \nabla F_{Ms}, \tag{29}$$

closed by quasineutrality and, for electromagnetic calculations, the appropriate field equations.

GX represents velocity space spectrally:

$$h_s = \sum_{\ell, m, k_x, k_y} \hat{h}_{s,\ell,m}(z,t) e^{i(k_x x + k_y y)} H_m\left(\frac{v_\parallel}{v_{ts}}\right) L_\ell\left(\frac{v_\perp^2}{v_{ts}^2}\right) F_{Ms}. \tag{30}$$

Inputs. The normal inputs are:

- magnetic geometry, commonly derived from VMEC,
- local profiles and gradients,
- collisionality, beta, flow shear, and numerical resolution.

Outputs. These codes provide growth rates, mode frequencies, nonlinear heat and particle fluxes, fluctuation spectra, and field energies. For workflow purposes the most important outputs are the saturated heat and particle fluxes.

How these outputs are used. Those fluxes are precisely what a transport solver needs. They are also frequently used directly as optimization objectives. This is why GX is so useful inside optimization loops.

4.8 Trinity3D: whole-device profile evolution

What Trinity3D does. Trinity3D advances radial density and temperature profiles using fluxes supplied by turbulence and neoclassical submodels [13].

Governing equations. In continuous form, the model is a 1D system of conservation laws:

$$\frac{\partial n_s}{\partial \tau} + \mathcal{G}(\rho)\frac{\partial F_{n,s}}{\partial \rho} = S_{n,s}, \tag{31}$$

$$\frac{\partial p_s}{\partial \tau} + \mathcal{G}(\rho)\frac{\partial F_{p,s}}{\partial \rho} = \frac{2}{3}S_{p,s}. \tag{32}$$

The current implementation advances a stacked profile vector y with an implicit linearized update:

$$(d_1 I + \alpha\Psi)\, y^{m+1} = -d_0 y^m - d_{-1} y^{m-1} + \alpha\Psi y^m - \alpha\left[G(F^+ - F^-) - S\right] - (1-\alpha)\left[G(F_m^+ - F_m^-) - S_m\right]. \tag{33}$$

Inputs and outputs. The code reads a TOML input specifying geometry, species profiles, source models, and transport submodels. In stellarator examples, the geometry is often VMEC, turbulence comes from GX, and neoclassical fluxes can come from SFINCS. The outputs are updated profiles, flux histories, and the coupled sub-run artifacts.

How Trinity3D fits in the workflow. This is typically the stage that turns local transport coefficients into a global profile prediction and hence into an updated pressure profile for the next equilibrium iteration.

4.9 NEOPAX: a reduced JAX neoclassical transport loop

What NEOPAX does. NEOPAX builds a reduced neoclassical transport model from a MONKES-style database $D_{ij}(r, \nu, E_r)$ and evolves profiles with a JAX-native ODE solve [32].

Governing equations. The thermal transport coefficients are assembled into an L_{ij} matrix, from which particle flux, heat flux, and parallel flow are computed:

$$\Gamma_a = -n_a\left(L_{11}A_1 + L_{12}A_2 + L_{13}A_3\right), \tag{34}$$

$$Q_a = -T_a n_a\left(L_{21}A_1 + L_{22}A_2 + L_{23}A_3\right), \tag{35}$$

$$U_{\|a} = -n_a\left(L_{31}A_1 + L_{32}A_2 + L_{33}A_3\right). \tag{36}$$

The current implementation also forms an ambipolar electric-field source from the charge-flux imbalance, schematically

$$S_{E_r} \propto -\Gamma_e + \Gamma_D + \Gamma_T, \tag{37}$$

and integrates the resulting radial evolution equations with `diffrax`.

Inputs and outputs. Inputs are a MONKES-style D_{ij} database, geometry, species profiles, and edge conditions. Outputs include E_r, neoclassical particle and heat fluxes, and profile evolution histories.

How NEOPAX fits in the workflow. It can replace a more expensive full transport loop when a differentiable, reduced-order neoclassical model is preferable.

5 Which quantities are optimization targets and which are transport inputs?

This distinction is easy to miss and is crucial when building a black-box workflow:

- **Geometry metrics** such as aspect ratio, boundary smoothness, or target rotational transform are equilibrium-stage objectives.
- **Boozer-spectrum metrics** such as dominant symmetry-breaking B_{mn} harmonics are usually optimization objectives and diagnostics.
- **Effective ripple** ϵ_{eff} is usually a direct objective or screening metric, not a transport variable that Trinity3D advances.
- **SFINCS fluxes** are transport variables: they can directly feed Trinity3D or another transport solver.
- **GX / GENE heat fluxes** are both optimization objectives and transport inputs.
- **Bootstrap current** is a transport output that can become a new equilibrium input.

The user-supplied Mermaid chart reflects this correctly: some outputs are promoted to the next physics stage, while others are used only as optimization targets or screening diagnostics.

6 Why the final temperature and density profiles differ from the initial guesses

This is the central feedback mechanism in the workflow.

The equilibrium calculation requires a pressure and current profile, but those profiles are often only a convenient starting point. Once neoclassical and turbulent transport are evaluated, the radial particle and heat fluxes tell us that the original profiles were not generally in balance with the sources and losses.

The transport stage therefore updates:

- species densities $n_s(r)$,
- species temperatures $T_s(r)$,
- the ambipolar electric field $E_r(r)$,
- and sometimes the bootstrap-current contribution.

The total pressure is then

$$p(r) = \sum_s n_s(r) T_s(r), \tag{38}$$

so when the transport solver changes n_s and T_s, the equilibrium pressure profile changes too. If the bootstrap current changes appreciably, the current profile also changes, which can shift the equilibrium again.

Thus a consistent design loop is

$$(R_{mn}, Z_{mn}, p, j) \to \text{equilibrium} \to \text{Boozer geometry} \to \text{transport fluxes} \to (n, T, E_r) \to (p_{\text{new}}, j_{\text{new}}) \to \text{equilibrium} \tag{39}$$

7 Examples from the literature

The workflow described here is not hypothetical. Several published studies already implement large parts of it:

- **Device-scale context: Wendelstein 7-X.** Wendelstein 7-X is the clearest experimental example of an optimized stellarator built around reduced neoclassical transport, good confinement, and steady-state operation. While it is not itself a single "workflow paper", recent operation summaries show the device-level goals that this code chain is trying to predict and optimize [7].
- **Precise quasi-symmetry optimization.** Landreman and Paul optimized stellarator magnetic fields for extremely precise quasi-symmetry, providing an archetypal example of equilibrium plus Boozer plus orbit-confinement analysis [19].
- **Direct turbulence optimization.** Kim et al. coupled DESC and GX directly inside an optimization loop, using nonlinear heat flux as an objective and Trinity3D for macroscopic profile evaluation [18]. This is very close to the workflow summarized in this repository.
- **First-principles profile prediction.** Bañón Navarro et al. coupled GENE-3D, KNOSOS, and a 1D transport solver to produce first-principles profile predictions for optimized stellarators [1]. Although the specific neoclassical and transport codes differ, the logic is the same as SFINCS + GENE / GX + Trinity3D or NEOPAX.
- **Fast neoclassical loops for optimization.** MONKES was explicitly developed to make monoenergetic neoclassical calculations fast enough for optimization loops [8].

Taken together, these studies show that the remaining challenge is primarily software engineering: standard interfaces, reproducible containers, and an orchestration layer.

8 What a future containerized black-box workflow should standardize

If the end goal is a one-click workflow, the most important design choice is not the graphical interface but the contract between stages. The minimum useful contracts are:

- an **equilibrium container** that accepts boundary and profile coefficients and emits a VMEC-compatible `wout` plus a small machine-readable summary,
- a **Boozer container** that accepts `wout` and emits `boozmn` plus a summary of symmetry-breaking harmonics,

- a **neoclassical container** that accepts Boozer geometry and profile gradients and emits ϵ_{eff}, bootstrap current, E_r, particle flux, and heat flux in a shared HDF5 schema,
- a **turbulence container** that accepts geometry plus local profiles and emits heat and particle fluxes in a shared NetCDF or HDF5 schema,
- a **transport container** that accepts those fluxes plus source models and emits updated profiles and fusion metrics.

That, rather than the physics itself, is the key engineering task for turning a research workflow into a black-box system.

References

[1] A. Bañón Navarro, A. Di Siena, J. L. Velasco, F. Wilms, G. Merlo, T. Windisch, L. L. LoDestro, J. B. Parker, and F. Jenko. First-principles based plasma profile predictions for optimized stellarators. *Nuclear Fusion*, 63(5):054003, 2023. doi: 10.1088/1741-4326/acc3af.

[2] A. H. Boozer. Transport and isomorphic equilibria. *Physics of Fluids*, 26(2):496–499, 1983. doi: 10.1063/1.864166.

[3] H.-S. Bosch and G. M. Hale. Improved formulas for fusion cross-sections and thermal reactivities. *Nuclear Fusion*, 32(4):611–631, 1992. doi: 10.1088/0029-5515/32/4/I07.

[4] R. Conlin, D. Panici, D. Dudt, E. Kolemen, et al. The desc stellarator code suite part ii: Perturbation and continuation methods. *Journal of Plasma Physics*, 89(5):905890512, 2023. doi: 10.1017/S0022377823000399.

[5] D. Dudt and E. Kolemen. Desc: A stellarator equilibrium solver. *Physics of Plasmas*, 27(10): 102513, 2020. doi: 10.1063/5.0020743.

[6] D. Dudt, D. Panici, R. Conlin, E. Kolemen, et al. The desc stellarator code suite part iii: Quasi-symmetry optimization. *Journal of Plasma Physics*, 89(4):905890405, 2023. doi: 10.1017/S0022377823000235.

[7] M. Endler, J. Baldzuhn, C. D. Beidler, H.-S. Bosch, S. Bozhenkov, B. Buttenschön, A. Dinklage, J. Fellinger, Y. Feng, G. Fuchert, Y. Gao, J. Geiger, O. Grulke, D. Hartmann, M. Jakubowski, R. König, H. P. Laqua, S. Lazerson, P. McNeely, D. Naujoks, U. Neuner, M. Otte, E. Pasch, T. Sunn Pedersen, V. Perseo, A. Puig Sitjes, K. Rahbarnia, N. Rust, O. Schmitz, A. Spring, T. Stange, A. von Stechow, Y. Turkin, E. Wang, and R. C. Wolf. Wendelstein 7-x on the path to long-pulse high-performance operation. *Fusion Engineering and Design*, 167:112381, 2021. doi: 10.1016/j.fusengdes.2021.112381.

[8] F. J. Escoto, J. L. Velasco, I. Calvo, M. Landreman, and F. I. Parra. Monkes: a fast neoclassical code for the evaluation of monoenergetic transport coefficients in stellarator plasmas. *Nuclear Fusion*, 64(7):076030, 2024. doi: 10.1088/1741-4326/ad3fc9.

[9] f0uriest. monkes repository. https://github.com/f0uriest/monkes, 2026. Accessed 2026-03-11.

[10] GENE team. Gene license and documentation page. https://genecode.org/license.html, 2026. Accessed 2026-03-11.

[11] T. Görler, X. Lapillonne, S. Brunner, T. Dannert, F. Jenko, F. Merz, D. Told, et al. The global version of the gyrokinetic turbulence code gene. *Journal of Computational Physics*, 230 (18):7053–7071, 2011. doi: 10.1016/j.jcp.2011.05.034.

[12] gyrokinetics. Gx repository. https://bitbucket.org/gyrokinetics/gx/src/gx/, 2026. Accessed 2026-03-11.

[13] gyrokinetics. Trinity3d repository and documentation. https://bitbucket.org/gyrokinetics/t3d/src/main/, 2026. Accessed 2026-03-11.

[14] Per Helander. Theory of plasma confinement in non-axisymmetric magnetic fields. *Reports on Progress in Physics*, 77(8):087001, 2014. doi: 10.1088/0034-4885/77/8/087001.

[15] Hidden Symmetries. booz_xform repository. https://github.com/hiddenSymmetries/booz_xform, 2026. Accessed 2026-03-11.

[16] Hidden Symmetries / Princeton Plasma Physics Laboratory. Vmec2000 repository. https://github.com/hiddenSymmetries/VMEC2000, 2026. Accessed 2026-03-11.

[17] S. P. Hirshman and J. C. Whitson. Steepest-descent moment method for three-dimensional magnetohydrodynamic equilibria. *Physics of Fluids*, 26(12):3553–3568, 1983. doi: 10.1063/1.864116.

[18] P. Kim, S. Buller, R. Conlin, W. Dorland, D. Dudt, R. Gaur, R. Jorge, E. Kolemen, M. Landreman, N. R. Mandell, and D. Panici. Optimization of nonlinear turbulence in stellarators. *Journal of Plasma Physics*, 90(2):905900210, 2024. doi: 10.1017/S0022377824000369.

[19] M. Landreman and E. Paul. Magnetic fields with precise quasisymmetry for plasma confinement. *Physical Review Letters*, 128(3):035001, 2022. doi: 10.1103/PhysRevLett.128.035001.

[20] M. Landreman, H. M. Smith, A. Mollén, and P. Helander. Comparison of particle trajectories and collision operators for collisional transport in non-axisymmetric plasmas. *Physics of Plasmas*, 21(4):042503, 2014. doi: 10.1063/1.4870077.

[21] Landreman group. Sfincs repository. https://github.com/landreman/sfincs, 2026. Accessed 2026-03-11.

[22] N. R. Mandell, W. D. Dorland, and M. Landreman. Laguerre-hermite pseudo-spectral velocity formulation of gyrokinetics. *Journal of Plasma Physics*, 84(1):905840108, 2018. doi: 10.1017/S0022377818000041.

[23] N. R. Mandell, W. D. Dorland, I. Abel, R. Gaur, P. Kim, M. Martin, and T. Qian. Gx: a gpu-native gyrokinetic turbulence code for tokamak and stellarator design. *Journal of Plasma Physics*, 90(4):905900402, 2024. doi: 10.1017/S0022377824000631.

[24] F. Merz. Gyrokinetic simulation of multimode plasma turbulence. *Computer Physics Communications*, 183(4):922–930, 2012. doi: 10.1016/j.cpc.2011.12.018.

[25] A. Mollén, M. Landreman, H. M. Smith, J. M. García-Regaña, and M. Nunami. Less collisional neoclassical transport due to tangential magnetic drifts in a commercial fusion reactor candidate. *Nuclear Fusion*, 58(9):096021, 2018. doi: 10.1088/1741-4326/aad5b8.

[26] V. V. Nemov, S. V. Kasilov, W. Kernbichler, and M. F. Heyn. Evaluation of 1/nu neoclassical transport in stellarators. *Physics of Plasmas*, 6(12):4622–4632, 1999. doi: 10.1063/1.873749.

[27] D. Panici, R. Conlin, D. Dudt, E. Kolemen, et al. The desc stellarator code suite part i: Quick and accurate equilibria computations. *Journal of Plasma Physics*, 89(4):905890406, 2023. doi: 10.1017/S0022377823000272.

[28] PlasmaControl. Desc repository. https://github.com/PlasmaControl/DESC, 2026. Accessed 2026-03-11.

[29] Princeton University. Stellopt / neo repository and documentation. https://github.com/PrincetonUniversity/STELLOPT, 2026. Accessed 2026-03-11.

[30] UWPlasma. booz_xform_jax repository. https://github.com/uwplasma/booz_xform_jax, 2026. Accessed 2026-03-11.

[31] UWPlasma. Neo_jax repository. https://github.com/uwplasma/NEO_JAX, 2026. Accessed 2026-03-11.

[32] UWPlasma. Neopax repository. https://github.com/uwplasma/NEOPAX, 2026. Accessed 2026-03-11.

[33] UWPlasma. sfincs_jax repository. https://github.com/uwplasma/sfincs_jax, 2026. Accessed 2026-03-11.

[34] UWPlasma. Spectrax-gk repository. https://github.com/uwplasma/SPECTRAX-GK, 2026. Accessed 2026-03-11.

[35] UWPlasma. vmec_jax repository. https://github.com/uwplasma/vmec_jax, 2026. Accessed 2026-03-11.

[36] P. Xanthopoulos, T. M. Bird, J. H. E. Proll, P. Helander, et al. Gene-3d: A global gyrokinetic turbulence code for stellarators. *Journal of Computational Physics*, 402:109694, 2020. doi: 10.1016/j.jcp.2020.109694.